What the thesis concludes, and what it leaves behind

The evidence divides by what is being measured before it divides by resource

Safety: a difference in kind

Safe Rust rejects all 7 injected memory-safety and data-race classes at compile time. The 7 boundary classes, from a deadlock to a forgotten lock guard, compile and misbehave exactly as they do in C++.

Rust eliminates data races, not race conditions, and aliasing errors, not lock-ordering errors.

Migration: the cost is human

An unmodified Rust core can be driven from C++ across a thin seam that concentrates every unsafe site, for a cost on a real operation bounded at 10.91% against the same operation called natively.

What migration costs is API design and the learning curve, not run time.

Performance: two different answers

End to end, an operation is 0.250% the work the language performs and the rest a cross-thread handoff. The two are equivalent at the median, and on a quiet host Rust's far tail sits lower.

Measured on the operation alone they are not equivalent: Rust is faster by about a factor of two, for reasons tracing to hash-map layout.

What this adds to the decision is an instrument, not a verdict

- A frozen specification
- A parity-tested pair
- A reproducible framework
- A catalogue of what the guarantee misses

Where it goes next

a canonical-configuration repeat. Bare metal already bought the far tail, the counters and the allocator's accounting.

- Production traces
- Further subsystems
- Call-site attribution
- Extend the catalogue
- Async runtimes
- Isolate the heaps

For stateful, concurrent, partitionable control-plane services of this kind, the choice between modern C++ and Rust is not a performance decision.